

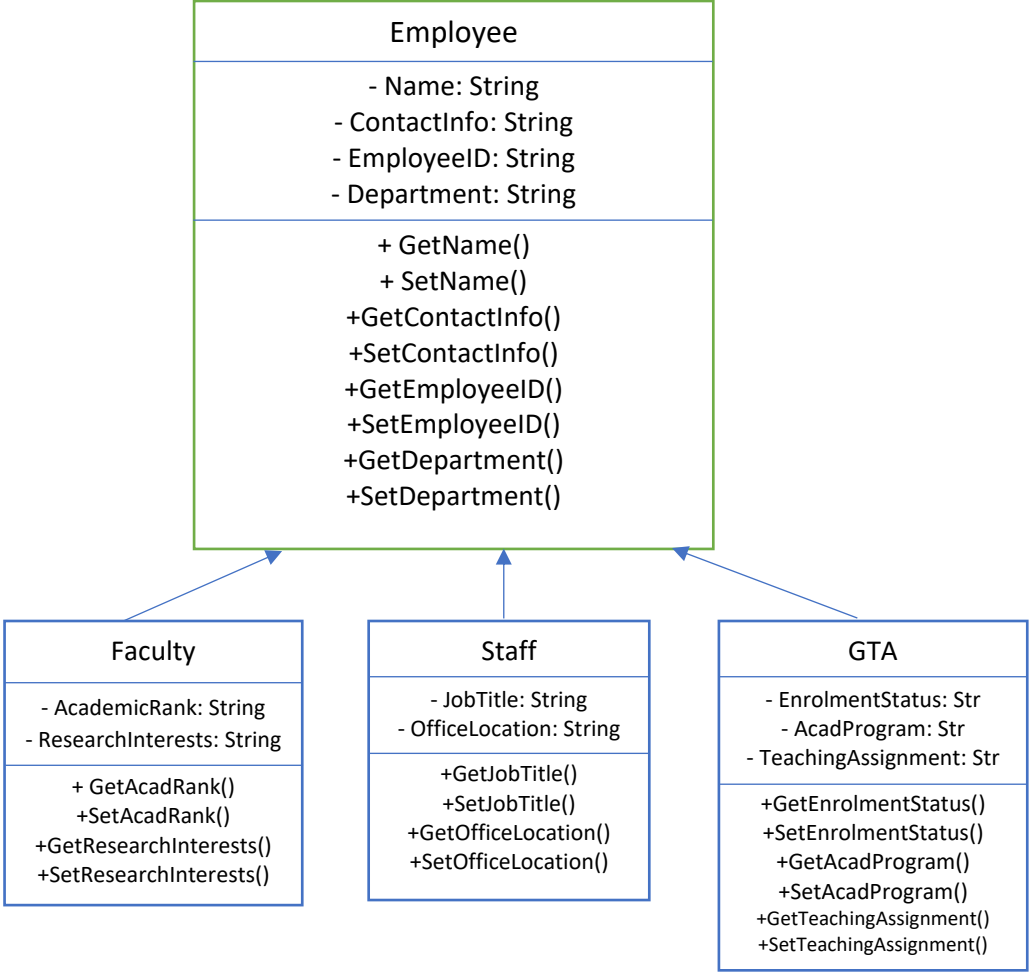
Y6 2024 H2 Comp TP Theory Paper 1 Solutions

1(a)	Function Call	arr	startIndex	endIndex	Return value
	1	[1, 2, 3, 4, 5, 6, 7]	3	5	$4*4 + K(arr, 4, 5)$ $= 16 + 41$ $= 57$
	2	[1, 2, 3, 4, 5, 6, 7]	4	5	$5 + K(arr, 5, 5)$ $= 5 + 36$ $= 41$
	3	[1, 2, 3, 4, 5, 6, 7]	5	5	$6*6 + K(arr, 6, 5)$ $= 36 + 0$ $= 36$
	4	[1, 2, 3, 4, 5, 6, 7]	6	5	0
1(b)	Function K recursively processes an array by summing up the square of even numbers and value of odd numbers, based on their positions indicated by startIndex and endIndex.				
1(c)	A run time error occurs when the program tries to access an element from a list using an invalid index, which happens when the index is out of bounds of the list. In the function call K([1, 2, 3, 4, 5, 6, 7], 3, 7), the error occurs because the function tries to access elements at indices beyond the length of the list, index 7, leading to an out-of-range index during execution.				
1(d)	<pre> FUNCTION K(arr, startIndex, endIndex) result <- 0 FOR i FROM startIndex TO endIndex IF arr[i] MOD 2 = 0 THEN result <- result + arr[i] * arr[i] ELSE result <- result + arr[i] ENDIF ENDFOR RETURN result ENDFUNCTION </pre>				
2(a)	$O(n^2)$ $O(n \log n)$				
2(b)	Bubble Sort sorts elements within the original array, while Merge Sort requires additional space and does not operate in-place due to its recursive divide-and-conquer approach.				
2(c)	<pre> PROCEDURE Sort_A(arr) n <- length of arr FOR i <- 0 TO n-1 noSwap <- True FOR j <- 0 to n-i-1 IF arr[j] > arr[j+1] THEN Swap arr[j] and arr[j+1] noSwap <- False ENDIF ENDFOR IF noSwap THEN Break ENDIF ENDFOR ENDPROCEDURE </pre>				

Y6 2024 H2 Comp TP Theory Paper 1 Solutions

	Implementing an early termination mechanism such as a flag in Bubble Sort can improve efficiency by stopping iterations if no swaps are made during a pass, indicating the array is already sorted. For example, for an already sorted array, the algorithm will only iterate the array once, during the first pass since there is no swaps made at all, resulting in $O(n)$, which is proportional to the number of elements.
3(a)	<pre> graph LR Customer[Customer] <--> Order[Order] Order <--> OrderLine[OrderLine] OrderLine <--> MenuItem[MenuItem] </pre>
3(b)	Customer (<u>CustomerID</u>, Name, Contact) Order (<u>OrderID</u>, Date, TotalAmount, <u>CustomerID</u>) OrderLine (<u>OrderID</u>, <u>MenuItemID</u>, Quantity) MenuItem (<u>MenuItemID</u>, Name, Description, Price) <u>Underline Primary key. Dash underline Foreign key.</u>
3(c)	<pre> SELECT c.*, SUM(o.TotalAmount) AS TotalSpent FROM Customer c INNER JOIN Order o ON c.CustomerID = o.CustomerID GROUP BY c.CustomerID ORDER BY TotalSpent DESC; </pre>
3(d)(i)	Schema: Structured rows and columns vs semi-structured, flexibility Scalability: Vertical vs Horizontal
3(d)(ii)	Diverse types of data expected, e.g. customers orders with varying sets of items Frequent changes in schema, e.g. include customers reviews High traffic website where customers can place orders, view menus, interact with services in real-time

Y6 2024 H2 Comp TP Theory Paper 1 Solutions

4(a)	 <pre> classDiagram class Employee { - Name: String - ContactInfo: String - EmployeeID: String - Department: String + GetName() + SetName() + GetContactInfo() + SetContactInfo() + GetEmployeeID() + SetEmployeeID() + GetDepartment() + SetDepartment() } class Faculty { - AcademicRank: String - ResearchInterests: String + GetAcadRank() + SetAcadRank() + GetResearchInterests() + SetResearchInterests() } class Staff { - JobTitle: String - OfficeLocation: String + GetJobTitle() + SetJobTitle() + GetOfficeLocation() + SetOfficeLocation() } class GTA { - EnrolmentStatus: Str - AcadProgram: Str - TeachingAssignment: Str + GetEnrolmentStatus() + SetEnrolmentStatus() + GetAcadProgram() + SetAcadProgram() + GetTeachingAssignment() + SetTeachingAssignment() } Employee < -- Faculty Employee < -- Staff Employee < -- GTA </pre>
4(b)(i)	Allows objects of different classes to respond to the same message (method call) in different ways. DisplayDetails() can display details of all attributes of different classes individually, which are distinct from other classes. For example, when DisplayDetails() is called, Staff class will display specific attributes of staff, while Faculty class will display specific attributes of faculty.
4(b)(ii)	Code Reuse: Each subclass can tailor its response to method calls, adapting behavior according to its distinct attributes. This promotes the reuse of code and facilitates diverse object behaviors within a unified interface. Enhanced Modularity and Extensibility: Promotes a modular approach by defining common behaviors in a superclass and allowing subclasses to specialise these behaviors. This modular design enhances the system's extensibility, enabling developers to add new functionality by creating new subclasses without modifying existing code. By maintaining a clear separation of concerns, polymorphism supports easier updates and modifications to specific behaviors, fostering code reliability and scalability over time.
5(a)	Registries: focus on allocation and maintenance of critical internet resources. Registrars: specialise in customer interaction and domain name registration services. Together, they contribute to the efficient and orderly operation of the domain name registration process, ensuring the accessibility and reliability of domain names on the internet.

Y6 2024 H2 Comp TP Theory Paper 1 Solutions

5(b)	Through a distributed hierarchical structure. When domain name entered into web browser, browser queries a DNS resolver (@ ISP) → checks cache for IP address. If not there, resolver contacts series of DNS servers up the hierarchy (root DNS server, TLD DNS, authoritative DNS) to resolve domain name. Once found, store IP address at domain look-up table for future use.
5(c)	Reliability Static: high reliability as they remain constant over time. This consistency simplifies remote access, server configuration, and tasks reliant on a specific IP. Dynamic: may pose reliability challenges as they can change each time a device reconnects to the network. This inconsistency can lead to issues with applications or services that rely on a fixed IP address for the device. Cost-effectiveness Static: may incur additional costs by ISP Dynamic: more cost-effective, as ISP can efficiently manage pool of addresses
5(d)	Connectionless Quick and independent packet deliver, e.g. UDP Without prior establishment of dedicated connection Faster transmission since no need for overhead associated with establishing & maintaining a connection. Scenario: real-time communication or quick data delivery, e.g. video streaming, may sacrifice some reliability compared to connection-oriented protocol. Connection-oriented Reliability and ordered delivery, e.g. TCP, packets arrive without errors Establish connection before delivery Scenario: reliable file transfer, web browsing, where data integrity and sequencing are critical.
6(a)	Acting as a barrier between an organization's internal network and external networks, such as the internet. Monitor and control incoming and outgoing network traffic based on predetermined security rules, known as filtering rules, to prevent unauthorized access and protect against various cyber threats. Establish a security perimeter , enforcing access controls and inspecting network packets to determine whether they should be allowed or blocked based on specified criteria. Help safeguard network resources , prevent unauthorised access to sensitive data, and mitigate the risk of cyber attacks
6(b)	Header information such as source and destination IP addresses, port numbers, and protocols. Packet payloads for specific keywords, patterns, or file types to enforce security policies.
6(c)	False positives, false negatives (incorrectly identifying benign activity as malicious, or fail to detect actual intrusions) Challenges in inspecting encrypted traffic (e.g. HTTPS, as payload content is encrypted and cannot be easily analyzed) Potential impact on network performance due to latency (additional processing and analysis required to inspect packets and apply intrusion prevention measures) Attackers may employ evasion techniques to bypass detection (techniques include fragmentation, obfuscation, and encryption, making it challenging for the systems to effectively identify and mitigate malicious activities)